IMPLEMENTAÇÃO DE INTERNAL DEVELOPER PLATFORM PARA GOVERNANÇA EM AMBIENTE ENTERPRISE

Eduardo Doval Zenardi Filho

SUMÁRIO EXECUTIVO

O presente projeto propõe a arquitetura e implementação de uma *Internal Developer Platform* (IDP) para uma organização multinacional de grande porte. Do ponto de vista dos usuários – compostos por desenvolvedores de software, engenheiros de dados e profissionais de segurança corporativa – o sistema atua como um portal centralizado de *self-service* para o provisionamento automatizado de infraestrutura e serviços em nuvem.

O sistema funcionará em um ambiente de nuvem distribuído globalmente (Microsoft Azure), servindo como a camada de abstração principal entre o código-fonte e a infraestrutura subjacente, como clusters Kubernetes (AKS) e bancos de dados. A principal função da plataforma é reduzir a carga cognitiva dos times de engenharia, oferecendo *templates* de arquitetura padronizados, ou *Golden Paths* (SKELTON; PAIS, 2019), que já contêm as melhores práticas de Infraestrutura como Código (IaC), CI/CD, monitoramento e governança de segurança embutidas.

PROBLEMATIZAÇÃO

Em ambientes corporativos de escala global, a proliferação de microsserviços e a adoção de múltiplas soluções em nuvem geram uma complexidade operacional insustentável (BEYER *et al.*, 2016). Atualmente, os times de desenvolvimento gastam uma parcela significativa de seu tempo configurando infraestrutura, gerenciando permissões de acesso e lidando com armazenamento de dados distribuídos, em vez de focar na entrega de valor de negócio.

Além da ineficiência e do gargalo operacional, essa falta de padronização arquitetural resulta em graves riscos de Segurança da Informação. Sem uma governança centralizada, aplicações são frequentemente implantadas com configurações de rede inadequadas, falta de rastreabilidade de logs e violações do princípio de privilégio mínimo. A ausência de um fluxo automatizado e validado dificulta a auditoria, a aplicação de *patches* de segurança em escala e a manutenção de uma postura de nuvem resiliente contra ameaças cibernéticas.

JUSTIFICATIVA E OBJETIVO GERAL

Justificativa

Um *Internal Developer Platform* (IDP) é, fundamentalmente, uma camada de autoatendimento (*self-service*) construída para abstrair a complexidade operacional da infraestrutura de nuvem (CNCF, 2026). Para uma organização *enterprise*, sua relevância é inestimável: ela atua como o motor central que traduz as intenções dos desenvolvedores em infraestrutura provisionada, permitindo que as equipes foquem na escrita de código de negócio em vez de gerenciar configurações de servidores, redes e permissões.

A implementação dessa plataforma justifica-se pela necessidade crítica de otimizar a entrega de software e garantir a padronização em um cenário corporativo globalizado e de alta complexidade. Atualmente, a fragmentação de ferramentas e a configuração manual geram uma alta carga cognitiva para as equipes, resultando em atrasos no *time-to-market* de novos produtos e na elevação dos custos operacionais.

Para o negócio, o impacto desta intervenção é estratégico e transformador. O IDP centraliza a governança corporativa, assegurando que todas as aplicações sigam padrões rigorosos de arquitetura e segurança desde o início do desenvolvimento (*security by design*). Além de automatizar o provisionamento em nuvem, a plataforma estabelece diretrizes claras para os ambientes de trabalho — padronizando, inclusive, o uso de assistentes de codificação baseados em inteligência artificial. Dessa forma, o IDP mitiga falhas humanas, reduz vulnerabilidades sistêmicas e permite que as equipes foquem exclusivamente na inovação, impulsionando a competitividade da organização.

Objetivo Geral

Propor a arquitetura e o modelo de implementação de um *Internal Developer Platform* (IDP) escalável e segura para o contexto de uma corporação multinacional, visando automatizar o provisionamento de infraestrutura *on-demand*, centralizar a governança e reduzir a carga cognitiva dos times, garantindo segurança na entrega contínua de software.

FUNDAMENTAÇÃO TEÓRICA

A construção de um IDP de nível *enterprise* requer a orquestração de tecnologias nativas de nuvem e ferramentas consolidadas para garantir escalabilidade, segurança e resiliência (CNCF, 2026). A arquitetura proposta fundamenta-se nos seguintes pilares:

Portal do Desenvolvedor, Catálogo de Serviços e *Golden Paths*

O Backstage (SPOTIFY, 2026) atua como a interface central da plataforma. Ele unifica o catálogo de software corporativo e fornece os *Golden Paths*. Um *Golden Path* (caminho pavimentado) é um conjunto de práticas, *templates* e ferramentas pré-configuradas e suportadas oficialmente pela organização para o desenvolvimento de software (SKELTON; PAIS, 2019). Sua importância reside em reduzir a fadiga de decisão técnica. Para estabelecer esses caminhos, a engenharia de plataforma deve tratar o IDP como um produto (*Platform as a Product*), colaborando continuamente com os desenvolvedores para criar *templates* que resolvam dores reais.

Nuvem Enterprise e Orquestração de Infraestrutura

O Microsoft Azure servirá como provedor base, com o AKS (*Azure Kubernetes Service*) como motor de orquestração de contêineres (MICROSOFT, 2026a). O Crossplane é adotado como o motor de Infraestrutura como Código (IaC). Ao invés de fluxos imperativos externos, o Crossplane estende a API do Kubernetes, permitindo provisionar recursos complexos do Azure nativamente (CROSSPLANE, 2026).

Automação, GitOps e Governança de IA

O Azure Pipelines atua como motor de CI, compilando o código, rodando testes automatizados, escaneando vulnerabilidades e publicando as imagens no *Azure Container Registry* (ACR). O ArgoCD atua como motor de CD (GitOps), garantindo que o estado da infraestrutura no AKS seja idêntico ao declarado nos repositórios. A automação também injeta nativamente arquivos de instrução/*skills* (como o padrão AGENT.md ou copilot-instructions.md, do GitHub Copilot) na raiz de cada projeto, governando o uso de IA local pelos engenheiros.

Mecanismos de Controle e Governança de IA

Ao utilizar um *Software Template* para criar um novo projeto, o IDP injeta automaticamente na raiz do repositório arquivos de instruções específicas (como *AGENT.md* ou *.github/copilot-instructions.md*).

Esses arquivos definem um *prompt* de sistema, fornecendo contexto adicional e regras de codificação para que assistentes como *GitHub Copilot* ou *Azure OpenAI* apliquem automaticamente em todas as interações de chat e geração de código no repositório. Neste arquivo é especificado:

1. As versões de bibliotecas aprovadas pela segurança;
2. O processo de desenvolvimento que a IA deve adotar (e.g.: *Test Driven Development* – TDD);
3. Os padrões de arquitetura (e.g.: *Clean Architecture*) que a IA deve seguir ao sugerir código; e
4. Restrições de segurança que impedem a IA de sugerir práticas obsoletas.

Dessa forma, o IDP não apenas facilita o uso da IA, mas atua como um filtro de governança, garantindo que o código gerado esteja em conformidade com as diretrizes corporativas antes mesmo do primeiro *build*.

Ciclo DevSecOps e Segurança (Cybersecurity)

A adoção do IDP e dos *Golden Paths* é o alicerce para materializar o ciclo DevSecOps. Ao invés de a segurança ser um gargalo no final do desenvolvimento, ela é injetada na origem (*Shift-Left*). O uso obrigatório de imagens de contêiner *distroless* da Chainguard é um exemplo prático dessa governança: os projetos já nascem livres de vulnerabilidades conhecidas e sem *shells* (CHAINGUARD, 2026). Complementarmente, o Kyverno atua como motor de *Policy as Code*, bloqueando implantações fora do padrão (KYVERNO, 2026), e o *Azure Key Vault* gerencia segredos de forma segura via *CSI Driver* (MICROSOFT, 2026b).

PERCURSO METODOLÓGICO DA INTERVENÇÃO

Nota Metodológica sobre o uso de Inteligência Artificial

Em conformidade com as diretrizes e boas práticas acadêmicas atuais, declara-se que a elaboração deste projeto contou com o apoio da ferramenta de inteligência artificial generativa Google Gemini. A IA foi utilizada exclusivamente como assistente para estruturação do escopo, *brainstorming* de soluções arquiteturais e organização textual das ideias. A autoria, as decisões arquiteturais da *stack* tecnológica (Azure, Crossplane, Kyverno), a curadoria do conteúdo e a validação teórica são de inteira responsabilidade do autor deste documento.

Arquitetura Lógica da Solução

Para garantir que o IDP seja flexível, confiável e resiliente a mudanças, a sua arquitetura lógica foi desenhada seguindo o princípio de forte separação de responsabilidades (SKELTON; PAIS, 2019). O sistema é modular o suficiente para que cada componente tenha um escopo único, garantindo alta coesão e baixo acoplamento.

Conforme ilustrado na Figura 1, o IDP opera através de um fluxo de automação que abstrai a complexidade operacional da infraestrutura de nuvem, centralizando o

desenvolvimento no padrão GitOps. O fluxo de criação de recursos é estruturado nas seguintes etapas:

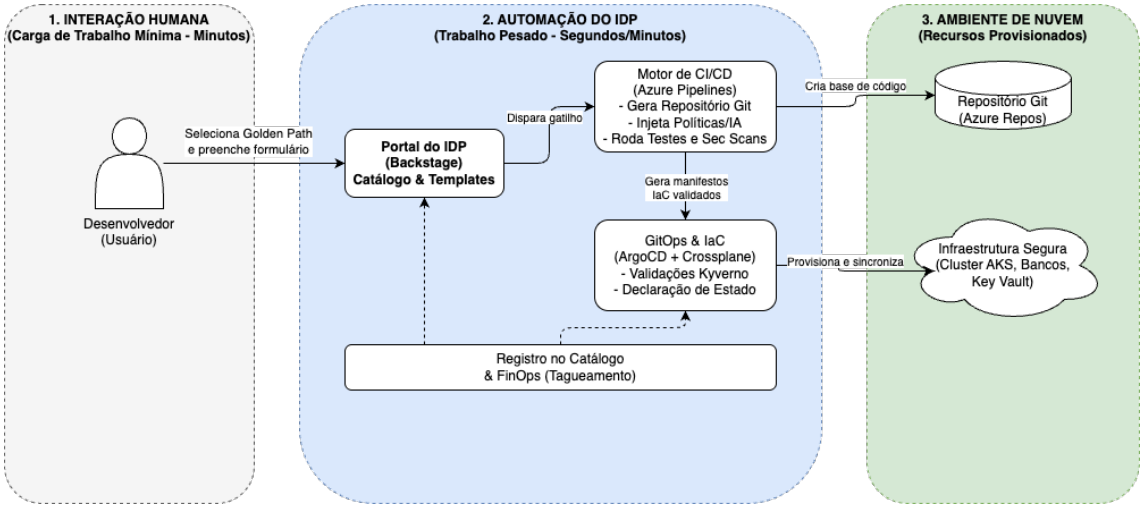

Figura 1: Fluxo de Automação e Arquitetura Lógica do IDP

1. **Interação e Portal da Plataforma:** O fluxo inicia-se com o Desenvolvedor (Usuário), que interage com o **Portal da Plataforma (Backstage)**. Utilizando o **Software Catalog** para descobrir padrões e os **Software Templates** para criar novos serviços via formulários pré-definidos, o desenvolvedor envia as intenções de provisionamento;
2. **Estado Desejado e Repositório Git:** O Portal executa a ação de **criar o serviço via template** e faz o *push* automático do código-fonte da aplicação e dos manifestos de infraestrutura para o **Repositório Git**. O repositório torna-se, assim, a fonte única da verdade sobre o estado desejado de todo o sistema;
3. **Orquestração e Agente GitOps:** O núcleo da orquestração reside no **Agente GitOps (ArgoCD + Crossplane)**. Este componente atua como o *Control Plane*, monitorando continuamente o Repositório Git e realizando a **sincronização** automática sempre que houver divergências entre o estado declarado no Git e o estado atual na nuvem;
4. **Execução de Workload e Infraestrutura:** Com base nos manifestos sincronizados, o agente bifurca a execução para provisionar os recursos de forma isolada e modular:

- Implantação da **carga de trabalho (*workload*)**, como *pods* e microsserviços, diretamente no **Cluster Kubernetes (AKS)**;
- **Provisionamento de Infraestrutura gerida** (como Bancos de Dados, filas, Key Vault, etc.) na **Infraestrutura Azure**, utilizando o Crossplane para traduzir manifestos do Kubernetes em chamadas de API nativas da Azure.

Fases de Implementação e Estratégia de Implantação

Para garantir a viabilidade e a aceitação do IDP na organização, a estratégia de implantação seguirá um modelo incremental dividido em grandes fases (*Milestones*):

- **Fase Piloto (*MVP*):** O projeto iniciará com o desenvolvimento de um Produto Mínimo Viável (*MVP*) em um ciclo curto de dois meses. O foco será validar as funcionalidades essenciais de provisionamento e catálogo com um grupo reduzido de desenvolvedores. Este *MVP* será apresentado para gerentes, times e lideranças técnicas (*Heads*) para demonstração de valor e obtenção de aprovação para escala.
- ***Milestone* 1 - Consolidação do Catálogo (Onboarding):** Com o projeto aprovado, o primeiro grande marco será o povoamento do *Software Catalog*. Nesta fase, será realizado o *onboarding* de produtos, APIs e softwares legados e atuais. A plataforma disponibilizará interfaces via linha de comando (*CLI*) e interface gráfica (*UI*) para que os responsáveis registrem seus ativos, permitindo que a plataforma ajude na manutenção e operação centralizada no Backstage.
- ***Milestone* 2 - Desenvolvimento de Templates Estratégicos:** O segundo marco consistirá em rodadas de alinhamento com os *Product Owners* (POs) para identificar as demandas reais dos times. O objetivo é desenvolver novos *templates* de *Golden Paths* que estejam alinhados com as necessidades de negócio e requisitos técnicos da organização.
- ***Milestone* 3 - Padronização Global e Governança:** O marco final focará na padronização rigorosa de todos os projetos sob a gestão do IDP. Isso inclui a obrigatoriedade de esteiras de CI/CD padronizadas, a aplicação de *guardrails* de segurança automáticos e a garantia de que as aplicações operem sem

vulnerabilidades conhecidas (CVEs), consolidando a cultura DevSecOps na empresa. As ferramentas que irão garantir estes *guardrails* são o SonarQube para garantir qualidade de código e segurança e Trivy para sondar vulnerabilidades e má-configurações.

Análise e Mitigação de Riscos

No que tange à análise e mitigação de riscos da plataforma, o primeiro desafio identificado é a possível rejeição por parte dos desenvolvedores. Como estratégia de mitigação, propõe-se tratar o IDP como um produto, envolvendo os usuários finais no desenho dos *templates*. Essa abordagem colaborativa é fundamental para garantir uma efetiva redução do tempo de setup local, diminuindo o atrito e incentivando a adoção orgânica da ferramenta pelas equipes de engenharia.

Um segundo risco crítico refere-se à falha na padronização de imagens seguras. Para mitigar essa vulnerabilidade e assegurar a integridade do ecossistema, utiliza-se imagens de container *Distroless* da Chaingard.

O terceiro risco seria implementar recursos Kubernetes sem seguir as melhores práticas, deixando a aplicação vulnerável. A implementação de uma ferramenta *Policy as Code*, como o Kyverno, garante de forma automatizada que o princípio de *Secure by Default* seja rigorosamente respeitado em todas as implantações (KYVERNO, 2026).

RESULTADOS ESPERADOS

A implementação do IDP deverá promover uma transformação na cultura de engenharia, na postura de segurança e na eficiência financeira da organização. Espera-se alcançar os seguintes resultados:

- **Métricas de Aceleração de Entrega:** Com a adoção dos *Golden Paths*, estima-se um ganho operacional sistêmico. Projeta-se que a implementação resulte em uma aceleração de até 30% no *lead time* para a entrega de novos

projetos, eliminando as esperas tradicionais associadas à abertura de chamados para operações (BEYER *et al.*, 2016);

- **Catálogo Unificado e Economia Financeira:** O catálogo fornecerá uma visibilidade arquitetural centralizada, evitando que times desenvolvam o mesmo software e compitam internamente. A mitigação desse retrabalho traduz-se em economia expressiva de orçamento de TI;
- **Governança Financeira (FinOps):** Através do tagueamento automatizado, a empresa terá uma noção exata de seus gastos (*cloud spend* e *workforce*), permitindo ajustes proativos (BEYER *et al.*, 2016);
- **Maximização da Segurança com Imagens *Distroless*:** A padronização das esteiras de CI/CD garantirá aplicações *secure by default* (CHAINGUARD, 2026). Espera-se uma redução drástica na superfície de ataque e otimização do tráfego de rede.

REFERÊNCIAS

BEYER, B. *et al*. **Site Reliability Engineering**: How Google Runs Production Systems. Sebastopol: O'Reilly Media, 2016.

CHAINGUARD. **Getting Started with Distroless Container Images?**: Securing the Software Supply Chain. [S. l.]: Chainguard, 2026. Disponível em: https://edu.chainguard.dev/chainguard/chainguard-images/about/getting-started-distroless/. Acesso em: 14 mar. 2026.

CNCF. **Cloud Native Landscape**. San Francisco: Cloud Native Computing Foundation, 2026. Disponível em: https://landscape.cncf.io/. Acesso em: 14 mar. 2026.

CROSSPLANE. **The Cloud Native Control Plane**. [S. l.]: Crossplane, 2026. Disponível em: https://crossplane.io/docs/. Acesso em: 14 mar. 2026.

KYVERNO. **Kubernetes Native Policy Management**. [S. l.]: Kyverno, 2026. Disponível em: https://kyverno.io/docs/. Acesso em: 14 mar. 2026.

MICROSOFT. **Arquitetura do Azure Kubernetes Service (AKS)**. Redmond: Microsoft Learn, 2026a. Disponível em: https://learn.microsoft.com/pt-br/azure/aks/concepts-clusters-workloads. Acesso em: 14 mar. 2026.

MICROSOFT. **Integração do Azure Key Vault com o AKS via Secrets Store CSI Driver**. Redmond: Microsoft Learn, 2026b. Disponível em: https://learn.microsoft.com/pt-br/azure/aks/csi-secrets-store-driver. Acesso em: 14 mar. 2026.

SKELTON, M.; PAIS, M. **Team Topologies**: Organizing Business and Technology Teams for Fast Flow. Portland: IT Revolution Press, 2019.

SPOTIFY. **Backstage**: An open platform for building developer portals. Estocolmo: Spotify, 2026. Disponível em: https://backstage.io/docs/overview/what-is-backstage. Acesso em: 14 mar. 2026.